

Procesamiento Masivo de Datos

Profesor: Alex Di Genova

Integrantes: Juan I. Riquelme y Martín Azua

Tarea 1

Procesamiento de archivos y mapeo probabilístico de lecturas

Este informe presenta las soluciones de procesamiento de datos con comandos Linux y describe un mapeador probabilístico de lecturas implementado en AWK, C secuencial y C con POSIX Threads. Los resultados experimentales se obtienen de los siete archivos de registro adjuntos: seis corresponden a *Escherichia coli* y uno a una ejecución con referencia humana.

1 P1. Procesamiento de datos con comandos Linux

Los comandos se ejecutan desde el directorio que contiene los archivos de entrada. Para `worldcitiespop.csv.gz`, se consideran los campos país, nombre normalizado de la ciudad, nombre de presentación, región y población en las columnas 1 a 5, respectivamente. Se omite el encabezado y se utiliza la coma como separador. Los ejercicios de `matrix.txt` suponen una matriz rectangular de enteros separados por espacios; se omiten líneas vacías. Se presentan comandos reproducibles, sin atribuir resultados numéricos a estos archivos, que no forman parte de los registros analizados.

1. Promedio de habitantes por ciudad y país.

Se agrupan los registros por país y nombre normalizado de ciudad. El promedio considera únicamente las entradas con población informada.

```
zcat worldcitiespop.csv.gz | awk -F',' '
NR > 1 && $5 != "" {
    clave = $1 SUBSEP $2
    suma[clave] += $5
    cantidad[clave]++
}
END {
    print "Pais,Ciudad,Promedio_habitantes"
    for (clave in suma) {
        split(clave, partes, SUBSEP)
        printf "%s,%s,%.2f\n", partes[1], partes[2],
            suma[clave] / cantidad[clave]
    }
}'
```

`zcat` descomprime el contenido hacia la salida estándar; no es necesario descomprimir el archivo previamente. La agrupación solicitada combina ciudades homónimas del mismo país; para distinguirlas por región, habría que incorporar la columna 4 a la clave.

2. Diez ciudades con mayor población registrada.

AWK extrae población, ciudad y país. Luego se ordena por población de mayor a menor y se muestran los primeros diez registros.

```
zcat worldcitiespop.csv.gz |  
awk -F',' 'BEGIN { OFS = "\t" }  
NR > 1 && $5 != "" { print $5 + 0, $3, $1 }' |  
sort -k1,1nr |  
head -n 10
```

3. Porcentaje de ciudades sin población informada por país.

El denominador incluye todos los registros del país. El numerador cuenta aquellos cuyo campo de población está vacío.

```
zcat worldcitiespop.csv.gz | awk -F',' '  
NR > 1 {  
    total[$1]++  
    if ($5 == "") faltantes[$1]++  
}  
END {  
    print "País, Porcentaje_sin_poblacion"  
    for (pais in total)  
        printf "%s, %.2f%%\n", pais,  
            100 * faltantes[pais] / total[pais]  
}'
```

4. Habitantes registrados y ciudad más poblada de América del Sur.

Se filtran los códigos de países y territorios sudamericanos, se suman sus poblaciones registradas y se identifica la ciudad con el valor máximo. La suma describe el contenido del archivo: no equivale necesariamente a la población oficial del continente ni imputa los valores faltantes.

```
zcat worldcitiespop.csv.gz | awk -F',' '  
NR > 1 &&  
tolower($1) ~ /^(ar|bo|br|cl|co|ec|fk|gf|gy|pe|py|sr|uy|ve)$/ &&  
$5 != "" {  
    poblacion = $5 + 0  
    habitantes += poblacion  
    if (!hay_datos || poblacion > maximo) {  
        maximo = poblacion  
        ciudad = $3  
        pais = $1  
    }  
    hay_datos = 1  
}  
END {  
    if (hay_datos) {  
        printf "Habitantes registrados: %.0f\n", habitantes  
        printf "Ciudad mas poblada: %s (%s), %.0f habitantes\n",  
            ciudad, pais, maximo  
    } else print "No hay registros con poblacion informada."  
}'
```

5. Mínimo y máximo de cada columna.

Los extremos se inicializan con la primera fila no vacía, por lo que el procedimiento también funciona si todos los valores son negativos.

```
awk '
NF > 0 {
    filas++
    if (filas == 1) columnas = NF
    for (i = 1; i <= NF; i++) {
        valor = $i + 0
        if (filas == 1 || valor < minimo[i]) minimo[i] = valor
        if (filas == 1 || valor > maximo[i]) maximo[i] = valor
    }
}
END {
    print "Columna Minimo Maximo"
    for (i = 1; i <= columnas; i++)
        print i, minimo[i], maximo[i]
}' matrix.txt
```

6. Inversión del orden de las columnas.

Se recorre cada fila desde su último campo hasta el primero, conservando un separador entre valores y un salto de línea al terminar. Para una matriz de 50 columnas, el orden resultante es 50, 49, ..., 1.

```
awk 'BEGIN { OFS = " " }
NF > 0 {
    for (i = NF; i >= 1; i--)
        printf "%s%s", $i, (i > 1 ? OFS : ORS)
}' matrix.txt
```

7. Filas cuyo promedio supera el promedio general.

La primera pasada obtiene el promedio de todos los elementos. La segunda calcula el promedio de cada fila y aplica el filtro. El archivo se indica dos veces para que AWK ejecute ambas pasadas.

```
awk '
FNR == NR {
    for (i = 1; i <= NF; i++) {
        total += $i
        cantidad++
    }
    next
}
NF > 0 && cantidad > 0 {
    suma = 0
    for (i = 1; i <= NF; i++) suma += $i
    if (suma / NF > total / cantidad) print
}' matrix.txt matrix.txt
```

8. Cantidad de números impares.

Un entero es impar cuando su resto al dividirlo por 2 es distinto de cero; la condición también incluye los impares negativos.

```
awk '
{
    for (i = 1; i <= NF; i++)
        if ($i % 2 != 0) impares++
}
END { print "Cantidad de numeros impares:", impares + 0 }
}' matrix.txt
```

9. Cantidad de números divisibles por 4 y por 9.

Se informan tres conteos: divisibles por 4, divisibles por 9 y divisibles por ambos simultáneamente. La última condición equivale a ser divisible por 36; el valor cero satisface las tres condiciones.

```
awk '
{
    for (i = 1; i <= NF; i++) {
        if ($i % 4 == 0) por4++
        if ($i % 9 == 0) por9++
        if ($i % 4 == 0 && $i % 9 == 0) ambos++
    }
}
END {
    print "Divisibles por 4:", por4 + 0
    print "Divisibles por 9:", por9 + 0
    print "Divisibles por 4 y por 9:", ambos + 0
}' matrix.txt
```

10. Fila con la mayor suma de sus elementos.

Se conserva la primera fila que alcanza la suma máxima. La inicialización con una fila válida evita asumir que el máximo es positivo.

```
awk '
NF > 0 {
    suma = 0
    for (i = 1; i <= NF; i++) suma += $i
    if (!hay_filas || suma > maxima) {
        maxima = suma
        numero = NR
        contenido = $0
    }
    hay_filas = 1
}
END {
    if (hay_filas) {
        print "Numero de fila:", numero
        print "Suma maxima:", maxima
        print "Contenido de la fila:", contenido
    } else print "El archivo no contiene filas numericas."
}' matrix.txt
```

2 P2. Mapeo probabilístico mediante semillas

2.1 Objetivo y criterios del algoritmo

El objetivo es estimar la posición de una lectura a partir de coincidencias de semillas de longitud k , o k -mers. El procedimiento acumula votos por posición candidata; no utiliza programación dinámica, distancia de edición, verificación base a base ni un alineador externo. Por esta razón, un `hit` representa una asignación del algoritmo y no demuestra por sí solo que la posición sea biológicamente correcta.

Las versiones en AWK, C secuencial y C con POSIX Threads comparten los siguientes criterios:

1. Se utiliza únicamente la hebra directa, sin calcular el complemento reverso.
2. En las ejecuciones documentadas, la firma auxiliar está desactivada (`fingerprint=0`). No se verifica la identidad de las semillas que acceden a una misma celda.
3. Cada celda conserva la primera posición insertada y su secuencia de referencia. Su contador aumenta con cada inserción.
4. El salto L se aplica tanto a la referencia como a las lecturas. Una celda solo aporta votos si su contador está entre 1 y el límite c . Todos los registros adjuntos utilizan $c = 1$.
5. Se aceptan mayúsculas y minúsculas. Las semillas con caracteres distintos de A, C, G y T se omiten; las bases ambiguas sí cuentan en las longitudes y coordenadas.
6. Una semilla no atraviesa límites entre secuencias FASTA. También se rechaza un candidato si la lectura completa quedaría fuera de su secuencia de referencia.
7. La salida utiliza el primer campo del identificador de la lectura. Las calidades FASTQ no intervienen en el puntaje; su longitud debe corresponder a la de la secuencia.

2.2 Construcción del índice

Sea R el número de secuencias FASTA, g_r la longitud de la secuencia r y

$$G = \sum_{r=1}^R g_r$$

la longitud total de referencia, incluidas las bases ambiguas. El diseño considera tamaños de índice

$$M = \left\lfloor \frac{G}{d} \right\rfloor, \quad d \in \{1, 2, 4, 8\},$$

y permite fijar M explícitamente, con $1 \leq M \leq \min(G, 4294967291)$. La comparación completa entre esos valores de d queda fuera de la evidencia adjunta: en *E. coli* se registró $M = G$, mientras que en humano se utilizó $M = 100\,000\,000$.

Para cada secuencia se insertan las semillas que comienzan en las posiciones $1, 1 + L, 1 + 2L, \dots$, siempre que quepan completas. El índice almacena contadores por celda y los datos de la primera

inserción. El contador puede crecer tanto por repeticiones de una semilla como por colisiones de hash; por ello, no debe interpretarse como un conteo exacto de copias biológicas.

Una celda con contador mayor que c se excluye de las consultas. Esta exclusión no implica necesariamente liberar la memoria que ocupa.

2.3 Función de hash y representación numérica

La función descrita en el diseño utiliza

$$b(A) = 1, \quad b(C) = 2, \quad b(G) = 3, \quad b(T) = 4, \quad B = 131, \quad P = 4294967291.$$

Para una semilla $s_1 \cdots s_k$, se inicia $h_0 = 0$ y se calcula

$$h_j = (131h_{j-1} + b(s_j)) \bmod P, \quad j = 1, \dots, k.$$

La celda, numerada desde cero, es $i = h_k \bmod M$. Equivalentemente,

$$h_k = \left(\sum_{j=1}^k b(s_j) 131^{k-j} \right) \bmod P.$$

La reducción modular mantiene acotados los valores intermedios. En efecto,

$$131h_{j-1} + b(s_j) \leq 131(P-1) + 4 = 562\,640\,714\,994 < 2^{40} < 2^{53}.$$

Estos enteros son representables exactamente en un número binario de doble precisión con 53 bits de significando. Esto permite reproducir la aritmética entera del hash en AWK bajo esa representación, y en C mediante `uint64_t`, sin codificar directamente las 31 bases en un entero de 62 bits. El hash es determinista, pero no elimina las colisiones.

Conceptualmente, una celda contiene los siguientes campos:

Campo	Tipo de referencia en C	Significado
Posición	<code>uint64_t</code>	Primera posición insertada
Contador	<code>uint32_t</code>	Número de inserciones en la celda
Referencia	<code>uint32_t</code>	Identificador de la secuencia FASTA

Con una estructura de 16 bytes, una tabla densa requeriría aproximadamente $16M$ bytes. Esta es una estimación del almacenamiento del índice, no de la memoria total del proceso: el tamaño real depende de la estructura, su alineación y los campos adicionales. AWK usa arreglos asociativos, cuya memoria tampoco se obtiene multiplicando directamente M por 16.

2.4 Candidatos, puntajes y formato de salida

Para una lectura de longitud n , se consultan semillas en desplazamientos $j = 0, L, 2L, \dots$ que cumplen $j + k \leq n$. Si una celda válida contiene la posición p de la referencia r , propone el inicio

$$a = p - j.$$

En esta notación, p y a están basados en 1, mientras que j es un desplazamiento basado en cero. El candidato se acepta únicamente si

$$a \geq 1 \quad \text{y} \quad a + n - 1 \leq g_r.$$

El puntaje cuenta las semillas consultadas que proponen el mismo par de referencia y posición:

$$\text{score}(r, a) = \#\{\text{semillas consultadas que proponen } (r, a)\}.$$

Se selecciona el mayor puntaje. Los empates se resuelven por la menor posición y, si persisten, por el orden de las secuencias en el FASTA. Incluir la referencia en la clave evita sumar votos de secuencias distintas. Una lectura sin votos se clasifica como **no-hit**; con al menos un voto, como **hit**.

La salida contiene una línea tabulada por lectura:

```
read_id status reference position score
```

Para una lectura sin asignación, los tres últimos campos son *, 0 y 0. Los resúmenes se escriben en la salida de error estándar y pueden guardarse en archivos `.log`, separados de las asignaciones.

2.5 Implementaciones

AWK secuencial. `mapper.awk` representa el índice y los votos mediante arreglos asociativos. Procesa las lecturas de forma secuencial, consulta las semillas, elige el mejor candidato y genera el resumen de asignaciones. Esta versión facilita la comparación de la lógica del algoritmo, aunque sus estructuras tienen un costo de memoria distinto al de una tabla densa en C. Los logs adjuntos permiten comparar sus resultados agregados, pero no incluyen tiempo ni memoria.

C secuencial. `mapper.c` construye el índice y procesa una lectura a la vez. Cada lectura genera sus propios candidatos y votos; después se aplica la regla de selección y desempate. El resumen registra tiempo, rendimiento, memoria residente, ocupación del índice y distribución de puntajes. Se utiliza como referencia temporal para las ejecuciones con hilos.

C con POSIX Threads. `mapper_pthreads.c` distribuye bloques de lecturas mediante un esquema productor–consumidor. El productor forma bloques de hasta X lecturas y los deposita en una cola de capacidad Y . Los trabajadores retiran bloques y consultan el índice, que permanece de solo lectura durante esta etapa.

La cola requiere un mutex y variables de condición para coordinar su acceso. En cambio, la lectura del índice compartido no requiere un mutex si su construcción ha terminado y ningún hilo lo modifica. Los votos de cada lectura se mantienen en estructuras propias del trabajador.

Los bloques terminados se identifican por su orden de entrada y se publican en ese orden mediante una sección protegida de salida. Finalmente, `pthread_join` permite esperar a que terminen los hilos antes de presentar las estadísticas finales. Los registros utilizan $X = 256$ y $Y = 8$, con cuatro trabajadores en *E. coli* y ocho en humano. La coincidencia de las estadísticas se examina en la siguiente sección; comprobar todas las asignaciones individuales requeriría además comparar los archivos de salida.

3 Resultados experimentales de los archivos .log

3.1 Fuentes y alcance de las mediciones

Se analizaron los siete registros de la tabla 1. Cada archivo representa una ejecución documentada. No se dispone de repeticiones por configuración, por lo que los tiempos se presentan como mediciones individuales y no como promedios. El apéndice A reproduce íntegramente su contenido.

Tabla 1: Archivos utilizados para elaborar los resultados.

Configuración	Archivo adjunto
E. coli, AWK, $k = 15$	<code>ecoli_awk_k15(2).log</code>
E. coli, AWK, $k = 31$	<code>ecoli_awk_k31.log(2).txt</code>
E. coli, C, $k = 15$	<code>ecoli_c_k15.log(2).txt</code>
E. coli, C, $k = 31$	<code>ecoli_c_k31.log(2).txt</code>
E. coli, pthreads, $k = 15$	<code>ecoli_pt_k15.log(3).txt</code>
E. coli, pthreads, $k = 31$	<code>ecoli_pt_k31.log(3).txt</code>
Humano, pthreads, $k = 31$	<code>human_pt(2).log</code>

La notación NR significa *no registrado*. Los tiempos se toman de `tiempo_real_seg`, el rendimiento de `reads_por_seg` y la memoria de `mem_residente_kb`. Se conserva la unidad de memoria indicada por el nombre del campo, kB, sin asumir que corresponda a un pico de consumo. El log no detalla qué etapas abarca el cronómetro ni cómo se obtiene la memoria; estas métricas se interpretan según lo reportado por el programa.

3.2 E. coli: configuración e índice

Las seis ejecuciones procesaron 415 598 lecturas sobre una única secuencia de referencia de 4 641 652 bases. Todas utilizaron $L = 10$, $c = 1$, $M = G = 4 641 652$ y firma desactivada. Las ejecuciones con POSIX Threads emplearon cuatro trabajadores, $X = 256$ y $Y = 8$.

Tabla 2: Estadísticas del índice de E. coli.

k	Celdas ocupadas	Ocupación reportada	Descartadas por c
15	439 347	0,0947	23 447
31	440 388	0,0949	22 826

La ocupación corresponde a la fracción de celdas ocupadas sobre M : aproximadamente 9,47% y 9,49%, respectivamente. Las tres implementaciones registran las mismas celdas ocupadas para cada k . El número de celdas descartadas aparece en los logs de C y pthreads; AWK no informa ese campo.

3.3 E. coli: asignaciones y distribución de puntajes

Tabla 3: Resultados comunes a AWK, C secuencial y C con pthreads.

k	Lecturas	hits	no-hits	hits (%)
15	415 598	390 743	24 855	94,02
31	415 598	386 478	29 120	92,99

Para un mismo k , las tres versiones coinciden en el total de lecturas, el número de asignaciones y el histograma completo de puntajes. Además, en los siete registros se verifican las identidades

$$\text{hits} + \text{no_hits} = \text{total_reads}, \quad \sum_{s \geq 0} H(s) = \text{total_reads},$$

donde $H(s)$ es el número de lecturas con puntaje s . También se cumple $H(0) = \text{no_hits}$. Esta consistencia respalda la coincidencia de las estadísticas agregadas; no prueba que cada identificador, referencia y posición de salida coincida.

Con $k = 15$ se obtuvieron 4 265 asignaciones más que con $k = 31$, una diferencia de 1,03 puntos porcentuales. Esta observación no permite concluir que una configuración sea más precisa: no se cuenta con posiciones verdaderas para evaluar las asignaciones.

Tabla 4: Concentración de puntajes en las ejecuciones registradas.

Conjunto	k	$H(0)$	$H(1)$	$H(1)/N$ (%)
E. coli	15	24 855	367 133	88,34
E. coli	31	29 120	363 649	87,50
Humano	31	1 777	96 590	96,59

En la tabla 4, N es el total de lecturas de cada ejecución, no el número de **hits**. Predominan las asignaciones con un único voto. Dado que la firma está desactivada y basta un voto para clasificar una lectura como **hit**, los porcentajes de asignación deben distinguirse de una medida de exactitud biológica.

3.4 E. coli: tiempo, rendimiento y memoria

Tabla 5: Mediciones de rendimiento en E. coli.

Versión	k	Hilos	Tiempo (s)	Lecturas/s	Memoria (kB)
AWK	15	1	NR	NR	NR
C secuencial	15	1	12,197	34 075,1	114 816
C pthreads	15	4	5,002	83 086,7	116 992
AWK	31	1	NR	NR	NR
C secuencial	31	1	14,762	28 153,3	114 944
C pthreads	31	4	4,455	93 282,4	116 736

Se define la aceleración respecto de C secuencial y la eficiencia por trabajador como

$$S_t = \frac{T_C}{T_{\text{pthreads},t}}, \quad E_t = \frac{S_t}{t}.$$

Se utiliza $t = 4$, el número de trabajadores reportado; el productor y las tareas de coordinación también pueden contribuir al tiempo total.

Tabla 6: Aceleración y eficiencia calculadas a partir de los tiempos.

k	Cálculo de S_4	Aceleración	Eficiencia (%)
15	12,197/5,002	2,44	60,96
31	14,762/4,455	3,31	82,84

Con cuatro trabajadores, el tiempo registrado se redujo aproximadamente un 58,99 % para $k = 15$ y un 69,82 % para $k = 31$. La memoria reportada aumentó 2 176 kB y 1 792 kB, respectivamente, respecto de C secuencial.

La aceleración es inferior a cuatro en ambas configuraciones. La lectura de datos, la coordinación, la publicación de resultados y las etapas secuenciales pueden limitar el beneficio del paralelismo; los registros no permiten cuantificar por separado esas contribuciones. Aunque $k = 31$ fue más lento en C secuencial y más rápido con pthreads en estas ejecuciones, una sola medición por caso no basta para establecer que ese comportamiento se repita de forma sistemática.

3.5 Referencia humana: ejecución con ocho trabajadores

El registro humano contiene un resumen completo de 100 000 lecturas. Su configuración fue $k = 31$, $L = 10$, $c = 1$, $M = 100\,000\,000$, ocho trabajadores, $X = 256$, $Y = 8$ y firma desactivada. El índice reporta solamente tres secuencias y 665 254 681 bases. Por lo tanto, los resultados se refieren a la referencia efectivamente cargada; no acreditan una ejecución sobre el conjunto completo de GRCh38.

Tabla 7: Resultados de `human_pt(2).log`.

Métrica	Valor reportado
Celdas ocupadas	45 475 422
Ocupación	0,4548
Celdas descartadas por c	12 854 768
Lecturas procesadas	100 000
<code>hits</code>	98 223
<code>no-hits</code>	1 777
Porcentaje de <code>hits</code>	98,22 %
Tiempo real (s)	270,930
Lecturas por segundo	369,1
Memoria residente (kB, según el log)	2 996 608

La ocupación registrada equivale aproximadamente al 45,48 %. El tiempo de 270,930 segundos corresponde a unos 4 minutos y 31 segundos. Como no se adjunta una ejecución secuencial humana equivalente, no es posible calcular su aceleración ni su eficiencia.

El 98,22 % de asignaciones no debe interpretarse como una tasa de alineamientos correctos. El tamaño y contenido de la referencia, el muestreo de semillas y las colisiones afectan la interpretación de los votos. Antes de extender la conclusión a GRCh38 completo, se debe verificar la integridad del FASTA y qué secuencias se cargaron.

3.6 Limitaciones y mediciones pendientes

Los archivos permiten comparar las tres implementaciones en estadísticas agregadas para E. coli, y medir la mejora temporal de pthreads frente a C secuencial. Para completar la evaluación quedan pendientes:

- Registrar tiempo y memoria de AWK con la misma metodología.
- Repetir cada configuración para estimar la variabilidad temporal.
- Medir otros números de trabajadores y los tamaños de índice $M = \lfloor G/d \rfloor$ con $d = 2, 4, 8$, además de $c = 2$.
- Comparar los archivos de asignaciones lectura por lectura y evaluar exactitud con posiciones de referencia conocidas.
- Verificar el alcance de la referencia humana y obtener una ejecución secuencial comparable.

Los logs no contienen el modelo de CPU, la configuración completa de memoria del equipo, las versiones del compilador ni sus opciones. Estos antecedentes deben acompañar futuras mediciones para facilitar su reproducción.

4 Conclusiones

Las tres implementaciones producen las mismas estadísticas de asignación y los mismos histogramas para E. coli en cada longitud de semilla evaluada. Con cuatro trabajadores, pthreads reduce el tiempo respecto de C secuencial y alcanza aceleraciones de 2,44 para $k = 15$ y 3,31 para $k = 31$, con un incremento pequeño de la memoria residente reportada.

La ejecución humana registra 100 000 lecturas en 270,930 segundos, con ocho trabajadores. Su alcance queda limitado a las tres secuencias cargadas. En ambos conjuntos, los `hits` describen asignaciones probabilísticas; demostrar exactitud y caracterizar la escalabilidad requiere las comprobaciones adicionales indicadas.

A Registros completos de las ejecuciones

Se transcriben los siete archivos sin modificar sus valores ni sus nombres de campo. Los saltos visuales de las líneas largas se añaden únicamente para facilitar la lectura. Los puntos decimales de esta sección se conservan tal como aparecen en los archivos originales.

A.1 E. coli, AWK, $k = 15$

ecoli_awk_k15(2).log

```
# Indice: nrefs=1 G=4641652 K=15 L=10 M=4641652 C=1 fingerprint=0
# Celdas ocupadas=439347 ocupacion=0.0947
# --- Resumen ---
# total_reads=415598 hits=390743 no_hits=24855 pct_hits=94.02%
# distribucion_de_scores: 0:24855 1:367133 2:700 3:231 4:129 5:125 6:139 7:118 8:99 9:105 10:83 11:97 12:112 13:126
    14:175 15:179 16:228 17:254 18:324 19:410 20:547 21:774 22:1067 23:1685 24:2564 25:3358 26:3755 27:3474
    28:2111 29:641
```

A.2 E. coli, AWK, $k = 31$

ecoli.awk_k31.log(2).txt

```
# Indice: nrefs=1 G=4641652 K=31 L=10 M=4641652 C=1 fingerprint=0
# Celdas ocupadas=440388 ocupacion=0.0949
# --- Resumen ---
# total_reads=415598 hits=386478 no_hits=29120 pct_hits=92.99%
# distribucion_de_scores: 0:29120 1:363649 2:244 3:198 4:139 5:174 6:131 7:136 8:175 9:166 10:178 11:191 12:212
    13:285 14:288 15:345 16:381 17:466 18:566 19:718 20:930 21:1340 22:1780 23:2294 24:2943 25:3304 26:2997 27:1741
    28:507
```

A.3 E. coli, C secuencial, $k = 15$

ecoli_c_k15.log(2).txt

```
# Indice: nrefs=1 G=4641652 K=15 L=10 M=4641652 C=1 fingerprint=0
# Celdas ocupadas=439347 ocupacion=0.0947 descartadas_por_c=23447
# --- Resumen ---
# total_reads=415598 hits=390743 no_hits=24855 pct_hits=94.02%
# tiempo_real_seg=12.197 reads_por_seg=34075.1 mem_residente_kb=114816
# distribucion_de_scores: 0:24855 1:367133 2:700 3:231 4:129 5:125 6:139 7:118 8:99 9:105 10:83 11:97 12:112 13:126
    14:175 15:179 16:228 17:254 18:324 19:410 20:547 21:774 22:1067 23:1685 24:2564 25:3358 26:3755 27:3474
    28:2111 29:641
```

A.4 E. coli, C secuencial, $k = 31$

ecoli_c_k31.log(2).txt

```
# Indice: nrefs=1 G=4641652 K=31 L=10 M=4641652 C=1 fingerprint=0
# Celdas ocupadas=440388 ocupacion=0.0949 descartadas_por_c=22826
# --- Resumen ---
# total_reads=415598 hits=386478 no_hits=29120 pct_hits=92.99%
# tiempo_real_seg=14.762 reads_por_seg=28153.3 mem_residente_kb=114944
# distribucion_de_scores: 0:29120 1:363649 2:244 3:198 4:139 5:174 6:131 7:136 8:175 9:166 10:178 11:191 12:212
13:285 14:288 15:345 16:381 17:466 18:566 19:718 20:930 21:1340 22:1780 23:2294 24:2943 25:3304 26:2997 27:1741
28:507
```

A.5 E. coli, C con pthreads, $k = 15$

ecoli_pt_k15.log(3).txt

```
# Indice: nrefs=1 G=4641652 K=15 L=10 M=4641652 C=1 fingerprint=0 hilos=4 X=256 Y=8
# Celdas ocupadas=439347 ocupacion=0.0947 descartadas_por_c=23447
# --- Resumen ---
# total_reads=415598 hits=390743 no_hits=24855 pct_hits=94.02%
# tiempo_real_seg=5.002 reads_por_seg=83086.7 mem_residente_kb=116992 nucleos_hilos=4
# distribucion_de_scores: 0:24855 1:367133 2:700 3:231 4:129 5:125 6:139 7:118 8:99 9:105 10:83 11:97 12:112 13:126
14:175 15:179 16:228 17:254 18:324 19:410 20:547 21:774 22:1067 23:1685 24:2564 25:3358 26:3755 27:3474
28:2111 29:641
```

A.6 E. coli, C con pthreads, $k = 31$

ecoli_pt_k31.log(3).txt

```
# Indice: nrefs=1 G=4641652 K=31 L=10 M=4641652 C=1 fingerprint=0 hilos=4 X=256 Y=8
# Celdas ocupadas=440388 ocupacion=0.0949 descartadas_por_c=22826
# --- Resumen ---
# total_reads=415598 hits=386478 no_hits=29120 pct_hits=92.99%
# tiempo_real_seg=4.455 reads_por_seg=93282.4 mem_residente_kb=116736 nucleos_hilos=4
# distribucion_de_scores: 0:29120 1:363649 2:244 3:198 4:139 5:174 6:131 7:136 8:175 9:166 10:178 11:191 12:212
13:285 14:288 15:345 16:381 17:466 18:566 19:718 20:930 21:1340 22:1780 23:2294 24:2943 25:3304 26:2997 27:1741
28:507
```

A.7 Referencia humana, C con pthreads, $k = 31$

human_pt(2).log

```
# Indice: nrefs=3 G=665254681 K=31 L=10 M=100000000 C=1 fingerprint=0 hilos=8 X=256 Y=8
# Celdas ocupadas=45475422 ocupacion=0.4548 descartadas_por_c=12854768
# --- Resumen ---
# total_reads=100000 hits=98223 no_hits=1777 pct_hits=98.22%
# tiempo_real_seg=270.930 reads_por_seg=369.1 mem_residente_kb=2996608 nucleos_hilos=8
# distribucion_de_scores: 0:1777 1:96590 2:489 3:158 4:206 5:183 6:210 7:205 8:106 9:54 10:17 11:4 12:1
```